

PassGauge

Password Intelligence and Parallel Computing:

Design, Multithreaded Implementation, and Deployment of a Desktop Application

Prepared by

Eng. Mohammad Ghaith al-Atrash

1 Nov 2026

Table of Contents

1. Introduction 1

1.1 Project Overview 1

1.2 Assignment Requirements 1

2. Theoretical Background 2

2.1 Concurrency and Parallelism 2

2.2 Threads, Synchronization, and Shared State 2

2.3 Race Conditions and Deadlocks 2

2.4 Speedup, Efficiency, and Amdahl's Law 3

3. System Architecture 4

3.1 High-Level Overview 4

3.2 Package Structure 4

3.3 Presentation Layer 5

4. Module 1 — Password Strength Analyzer (Complementary Feature) 5

4.1 Theoretical Basis 5

4.2 Implementation 6

4.3 Local Password Generation 6

5. Module 2 — Parallel Benchmark (Core Assignment Implementation) 7

5.1 The Workload: SHA-256 Brute-Force Search 7

5.2 Thread Management 8

5.3 Sequential Implementation 8

5.4 Parallel Implementation 9

5.5 Speedup and Efficiency Calculation 9

5.6 Dynamic Visualization and Historical Results 9

5.7 CSV Export and System Information 10

6. User Interface Design 10

6.1 Visual Theme 10

6.2 Interaction Details 10

7. Deployment 11

7.1 From Source to Runnable Artifact 11

7.2 Building a Self-Contained Runtime Image 11

7.3 Generating the Native Windows Installer 11

7.4 Distribution 12

8. Testing and Verification 12

9. Conclusion 14

References 14

1. Introduction

1.1 Project Overview

PassGauge is a two-module JavaFX desktop application developed to fulfill a parallel-programming assignment while also serving as a complete, usable tool in its own right. The assignment required selecting a computational problem that can be meaningfully divided across multiple threads, implementing it in both a single-threaded and a multi-threaded form, embedding the comparison inside a real application, and measuring the resulting speedup. PassGauge satisfies this through its Benchmark module, which brute-forces a password's SHA-256 hash sequentially and in parallel, and reports the timing difference between the two.

Alongside the assignment's core requirement, the application was extended with a complementary module — a local, privacy-preserving password strength analyzer — so that the final deliverable is not only a demonstration of a parallel algorithm, but a coherent, self-contained desktop tool centered on one theme: passwords, examined from two angles — how strong they are, and how quickly they can be found by brute force.

1.2 Assignment Requirements

The assignment specified three mandatory phases for the chosen parallel algorithm and its application:

1. Thread management — creating threads, dividing the workload across them, synchronizing shared state safely, and avoiding race conditions and deadlocks.
2. Algorithm implementation — the same computational problem implemented twice: a traditional single-threaded (sequential) version, and a multi-threaded (parallel) version.
3. Application integration — embedding the algorithm inside a complete, usable application that displays the execution time of both versions and computes the resulting speedup factor, Speedup = $T_{sequential} \div T_{parallel}$.

The specific algorithm family selected from the assignment's suggested list was parallel hashing, applied to the problem of testing password strength by generating and comparing millions of SHA-256 hashes per second — exactly the brief given for this category in the assignment sheet. The remainder of this report documents how each requirement was met, the theory underpinning the design decisions, and the process used to package and deploy the finished application.

2. Theoretical Background

2.1 Concurrency and Parallelism

Concurrency and parallelism are related but distinct concepts. Concurrency describes a program structured as multiple independent units of work that can be interleaved — on a single CPU core, this interleaving creates the appearance of simultaneity through rapid context-switching. Parallelism, by contrast, requires multiple units of work to execute at the literal same instant, which in practice requires multiple CPU cores. PassGauge's Benchmark module exploits true parallelism: the host machine's available processor cores (reported in the application's System Information line) each run one of the worker threads simultaneously, so the parallel pass is not merely concurrent but genuinely executes faster in wall-clock time.

2.2 Threads, Synchronization, and Shared State

A thread is an independent sequence of execution within a process, sharing the same memory space as every other thread in that process. This shared memory is what makes multithreading powerful — threads can cooperate on a single problem without the overhead of inter-process communication — but it is also the source of every classic concurrency hazard, because two threads reading and writing the same memory location without coordination produce undefined, non-deterministic results.

Synchronization is the set of techniques used to coordinate threads safely. The most common mechanism is a lock (or mutex), which forces threads to take turns accessing a shared resource. PassGauge deliberately avoids locks in its hot path — the per-candidate hash-and-compare loop that runs millions of times per second — because acquiring and releasing a lock on every iteration would itself become the performance bottleneck, undermining the very speedup the benchmark is trying to demonstrate. Instead, the application uses lock-free atomic variables (java.util.concurrent.atomic.AtomicBoolean and AtomicLong), which rely on hardware-level compare-and-swap instructions to guarantee safe concurrent updates without ever blocking a thread.

2.3 Race Conditions and Deadlocks

A race condition occurs when the correctness of a program depends on the relative timing of threads — for example, if two threads both check “is the password found?”, both see “no”, and both then declare themselves the winner. PassGauge avoids this specific race by using AtomicBoolean.compareAndSet(false, true) to claim the “found” flag: this operation is atomic at the processor level, so exactly one thread can ever successfully flip it from false to true, even if many threads attempt it in the same instant.

A deadlock occurs when two or more threads each hold a resource the other needs, and neither can proceed. Because the worker threads in ParallelCracker never acquire a lock or wait on another thread — each worker only reads a shared atomic flag and writes to its own local variables — there is no circular dependency for a deadlock to form. The only point of thread coordination is ExecutorService.invokeAll(...), which blocks the calling (UI-background) thread until all workers for the current candidate length have finished or stopped; this is a one-directional wait with no possibility of a cycle.

2.4 Speedup, Efficiency, and Amdahl's Law

Two metrics are used throughout the Benchmark module to quantify the benefit of parallelization:

Speedup

$$S = T_{sequential} / T_{parallel}$$

How many times faster the parallel run completed relative to the single-threaded baseline. A speedup of 1 means no benefit; a speedup of 4 on 4 threads would be perfect linear scaling.

Parallel Efficiency

$$E = (S / N) \times 100\%$$

Where N is the thread count used. Efficiency expresses how well the additional threads were actually utilized — 100% means every added thread contributed its full theoretical share of the speedup.

In practice, efficiency falls short of 100% and declines further as thread count increases, a phenomenon formally described by Amdahl's Law. The law states that if a fraction p of a program's work can be parallelized and the remaining fraction (1 − p) is inherently sequential, the maximum possible speedup from N processors is bounded:

Amdahl's Law

$$S(N) = 1 / [(1 - p) + p/N]$$

As N approaches infinity, speedup converges to 1/(1−p) — a hard ceiling set entirely by the sequential portion, no matter how many threads are added.

In PassGauge's case, the sequential portion includes starting and shutting down the thread pool, the atomic flag checks every worker performs on every iteration, and the brief coordination point at ExecutorService.invokeAll(...) between candidate lengths. This is why the Run History table and the Speedup/Efficiency chart (Section 5.6) typically show efficiency gradually decreasing as the thread count climbs from 1 through 16 — exactly the behavior Amdahl's Law predicts, and a key observation for the experimental discussion this report is required to include.

3. System Architecture

3.1 High-Level Overview

PassGauge follows a layered structure typical of a JavaFX desktop application: a declarative presentation layer (FXML + CSS) describes what the user sees, a single controller class wires that layer to application logic and handles all UI events, and the logic itself is split into two independent packages — one per module — so that the strength-analysis code and the benchmark code have no dependency on each other. Figure 1 summarizes this structure.

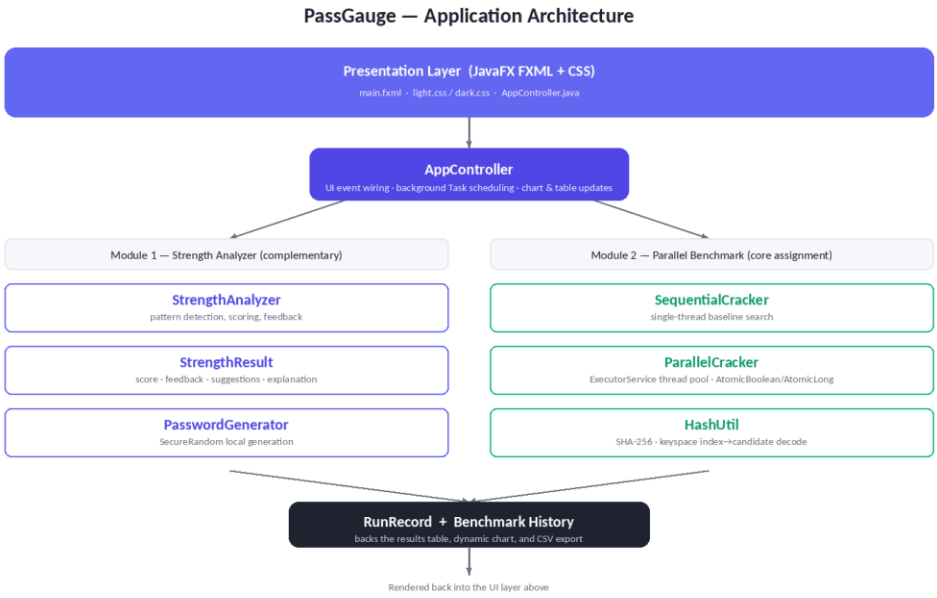


Figure 1 — PassGauge package and data-flow architecture.

3.2 Package Structure

The source tree is organized by responsibility rather than by module, with each Java package mapping to a single concern:

- **com.passgauge** — the Main class, the JavaFX application entry point; loads main.fxml and applies the default stylesheet.
- **com.passgauge.controller** — AppController (all UI wiring for both modules) and RunRecord (one benchmark result, backing the table, chart, and CSV export simultaneously).
- **com.passgauge.crack** — the parallel-programming core: Cracker (shared interface), SequentialCracker, ParallelCracker, and CrackResult.
- **com.passgauge.util** — HashUtil, providing SHA-256 hashing and the keyspace index-to-candidate decoder used by both crackers.

- **com.passgauge.strength** — the complementary module: StrengthAnalyzer, StrengthResult, and PasswordGenerator.

3.3 Presentation Layer

The user interface is declared in a single FXML document (main.fxml) using a BorderPane root: a slim gradient header holds the application identity and the dark-mode toggle, and the center region hosts a TabPane with one tab per module. Visual styling is kept entirely out of Java code and out of the FXML itself, living instead in two parallel stylesheets, light.css and dark.css, which share an identical set of CSS selectors but define different values for a set of JavaFX "looked-up color" variables (custom properties such as -accent and -text-primary declared once on the root pane). Toggling the theme swaps which stylesheet is attached to the Scene; because every component's color is expressed as a reference to one of these variables rather than a literal value, the entire interface re-themes correctly with no per-component logic required.

4. Module 1 — Password Strength Analyzer (Complementary Feature)

This module is a deliberate addition beyond the assignment's core requirement. It does not use multithreading — its purpose is complementary rather than computational: where the Benchmark module demonstrates how fast a weak password can be found, the Strength Analyzer explains, before an attack ever happens, why a given password would or would not hold up. Together the two modules form a complete narrative around password security rather than two unrelated demos.

4.1 Theoretical Basis

Password strength estimation in this module is modeled on the ideas popularized by the zxcvbn library (Wheeler, 2016): rather than scoring a password purely by length and character-class checklist rules (an approach well known to produce misleadingly high scores for passwords like "P@ssw0rd1"), the estimator looks for the specific patterns real attackers exploit — dictionary and common-password matches, sequential runs (abc, 123), keyboard-adjacency patterns (qwerty, asdf), repeated characters or repeated blocks, date-like substrings, and simple character substitutions ("leet speak", e.g. '@' for 'a'). A true zxcvbn implementation (such as the Java port zxcvbn4j) was considered; the project instead implements a smaller, self-contained analyzer following the same principles, for the practical reason discussed in Section 7.1 — the project uses no build tool, which makes managing an external dependency and its own transitive dependencies impractical without a dependency manager.

The analyzer combines two signals into a final 0–4 score (displayed as Very Weak through Strong): a raw entropy estimate, bits = length × $\log_2$(character-pool size), representing the theoretical size of the search space an attacker would face; and a penalty subtracted from that estimate for every predictable pattern detected, since a pattern-based password's effective search space is far smaller than its raw entropy suggests — an attacker checks "password123" long before exhausting a truly random 9-character space.

4.2 Implementation

StrengthAnalyzer.analyze(String password) performs this scan in a single pass and returns a StrengthResult: the numeric score, an estimated bit count, a list of feedback strings describing specific weaknesses found, a list of concrete suggestions, and a one-paragraph plain-English explanation generated from the score and detected weaknesses. Pattern detection is implemented with small dedicated routines rather than a single monolithic regular expression, for both correctness and explainability — each routine corresponds directly to one line of feedback the user sees:

- **hasSequentialRun()** scans for any run of 3+ consecutive characters whose character codes increase or decrease by exactly 1 (covers both alphabetic and numeric sequences in one pass).
- **containsRunOrReverse()** checks the password against a small set of known keyboard-row strings (qwertyuiop, asdfghjkl, etc.) and their reverses, at shrinking substring lengths.
- **hasRepeatedChar() / hasRepeatedBlock()** detect a single character repeated 3+ times, or a short substring immediately repeated (e.g. “abcabc”).
- **deLeet()** normalizes common substitutions (0→o, 1→l, 3→e, @→a, $→s, …) before checking the password against a built-in common-password set, so “p@ssw0rd” is still caught.

Crucially, StrengthAnalyzer never writes the password it analyzes anywhere — not to a log, not to a file, not even into this application's own benchmark history. The method takes a String, computes a result, and returns; no field in the class retains a reference to any password it has processed. This was a specific design requirement: strength analysis must be privacy-preserving and fully local, with nothing transmitted or retained after the analysis completes.

4.3 Local Password Generation

PasswordGenerator complements the analyzer with an on-device random password generator built on java.security.SecureRandom — a cryptographically strong random number source, in contrast to the predictable java.util.Random used for non-security-sensitive purposes elsewhere in typical applications. Generation guarantees at least one character from each of four classes (lowercase, uppercase, digit, symbol) by seeding the candidate list with one of each before filling the remainder of the requested length from the combined pool, then shuffling the full list so the guaranteed characters are not predictably positioned. As with analysis, nothing about a generated password is transmitted externally; generation happens entirely within the JVM process.

5. Module 2 — Parallel Benchmark (Core Assignment Implementation)

This module is the direct implementation of the assignment's three required phases.

5.1 The Workload: SHA-256 Brute-Force Search

The benchmark simulates an attacker who possesses only a password's SHA-256 hash — the standard way credentials are stored server-side — and must recover the original password by generating candidate strings, hashing each one, and comparing it against the target hash. This workload was selected because each candidate check is fully independent of every other: no candidate's outcome depends on any other candidate, which makes the problem embarrassingly parallel and an ideal, honest demonstration of multithreaded speedup, as opposed to a workload with heavy inter-thread dependencies that would understate the benefit of parallelism.

Rather than storing or generating candidate strings as an explicit list (infeasible in memory once the search space reaches millions or billions of entries), HashUtil.indexToCandidate(long index, char[] charset, int length) treats the character set as the digits of a mixed-radix number and decodes any integer index directly into the corresponding candidate string. This is the detail that makes partitioning the search space across threads possible at all: a worker thread can be handed a numeric range such as [2,000,000, 4,000,000) and jump straight to the start of its assigned range without generating or skipping over a single candidate that belongs to another thread.

```
public static String indexToCandidate(long index, char[] charset, int length) {
    char[] out = new char[length];
    int base = charset.length;
    for (int pos = length - 1; pos >= 0; pos--) {
        out[pos] = charset[(int) (index % base)];
        index /= base;
    }
    return new String(out);
}
```

HashUtil.indexToCandidate — decodes a numeric position directly into its candidate string.

The character set itself is not chosen manually by the user; it is derived automatically from the characters actually present in the password being tested (lowercase, uppercase, digit, and symbol classes are each included only if the password contains at least one character of that class). This guarantees the search space always actually contains the target password, while also simplifying the interface — the “will search for” chips shown under the password field update live as a visual confirmation of what charset is in effect.

5.2 Thread Management

Both crackers implement a shared Cracker interface, so the application can treat the sequential and parallel strategies interchangeably:

```
public interface Cracker {
    CrackResult crack(String targetHash, char[] charset, int maxLength,
                      int threadCount, AtomicLong hashCounter) throws InterruptedException;
}
```

ParallelCracker is where the assignment's thread-management requirement is met directly. For each candidate length (searched shortest-first), the keyspace of size charset.length^length is divided into threadCount contiguous, equally sized ranges, and a java.util.concurrent.ExecutorService fixed thread pool is handed one Callable task per range. Three synchronization mechanisms coordinate the pool:

- **AtomicBoolean found** — the single flag every worker polls between iterations and may set via compareAndSet(false, true). Because this operation is atomic at the hardware level, exactly one thread can ever win the race to set it, eliminating the “who found it first” race condition described in Section 2.3.
- **AtomicLong hashCounter** — incremented on every single hash attempt across all threads, providing a live, thread-safe running total that the UI polls roughly 60 times per second via a JavaFX AnimationTimer to display the live hash-rate counter during a run.
- **ExecutorService.invokeAll(tasks)** — blocks the calling thread until every worker for the current candidate length has either found the match or exhausted its assigned range, before the search proceeds to the next length. This is the only point of thread coordination in the entire algorithm, and because it is a one-directional wait (the caller waits on the pool; the pool never waits on the caller or on each other), no deadlock is possible.

No lock, synchronized block, or wait/notify pair appears anywhere in the hot path. This was a deliberate choice: with millions of iterations per second across multiple threads, even an uncontended lock's acquire/release overhead would be measurable, and would understate the true achievable speedup — precisely the number this benchmark exists to report accurately.

5.3 Sequential Implementation

SequentialCracker is the single-threaded baseline every parallel run is measured against. It performs exactly the same search — same character set, same length progression, same candidate-decoding logic — but with one thread and no shared mutable state beyond simple local counters, which is why it requires no synchronization at all. Keeping the sequential and parallel versions logically identical in every respect except thread count is what makes the resulting speedup measurement meaningful: any difference in execution time can be attributed to parallelism itself, not to an unrelated algorithmic difference between the two versions.

5.4 Parallel Implementation

The parallel search strategy, in full, for a single candidate length:

1. Compute the keyspace size for this length (charset.length raised to the power of length).
2. Determine the actual number of workers to use — min(threadCount, keyspace size) — so a tiny keyspace never spawns more threads than it has work to distribute.
3. Divide the keyspace into that many contiguous ranges and submit one Callable per range to the thread pool.
4. Each worker iterates its own range only, decoding each index to a candidate, hashing it, and comparing to the target — checking the shared found flag between iterations and stopping immediately if it is already true.
5. invokeAll() blocks until every worker for this length has returned; if found is now true, the search ends and the matching candidate (captured in an AtomicReference<String>) is returned; otherwise the loop proceeds to the next candidate length.

This range-partitioning approach is a textbook example of data parallelism — the same operation (hash-and-compare) applied independently to many pieces of a single, divisible dataset (the keyspace) — as distinct from task parallelism, where different threads would run fundamentally different operations.

5.5 Speedup and Efficiency Calculation

After both passes complete, AppController.showResults() computes:

Measured in the application

```
speedup = seqSeconds / parSeconds;  efficiency = (speedup / threads) × 100
```

Both figures are displayed immediately as metric tiles, and recorded as a RunRecord appended to the run history for later comparison.

5.6 Dynamic Visualization and Historical Results

Every completed run appends a RunRecord to an observable list backing three views at once: the Run History table, a dynamic bar chart, and the CSV export described in Section 5.7. The chart is “dynamic” in the sense required by the assignment brief — three chip buttons above it (Time / Speedup / Efficiency) switch which metric the bars represent, with the chart and its axis label rebuilt from the same underlying run history each time a different chip is selected, rather than requiring a fresh benchmark run for each view. Running the same password at thread counts of 1, 2, 4, 8, and 16 and comparing the resulting bars is the intended workflow for populating the experimental section of a report on this project, and is exactly what demonstrates the Amdahl's-Law effect discussed in Section 2.4.

5.7 CSV Export and System Information

The Export CSV button writes the complete run history to a file chosen via a standard JavaFX FileChooser, with one row per run (thread count, max length, sequential seconds, parallel seconds, speedup, efficiency percentage, and hashes per second), suitable for direct inclusion in a lab report or further analysis in a spreadsheet. A system-information line beneath the chart reports the host machine's available processor count (Runtime.getRuntime().availableProcessors()) and the running JVM's Java version (System.getProperty("java.version")) — context the assignment specifically asks the report to document, since the achievable speedup on any given machine is fundamentally bounded by its physical core count.

6. User Interface Design

6.1 Visual Theme

The interface follows a card-based layout: a slim gradient header (brand identity and the dark-mode toggle) sits above a two-tab body, with each tab's controls and results grouped into white (or, in dark mode, dark slate) rounded cards against a neutral page background. The palette — an indigo/violet accent (#6366F1 / #4F46E5) with a emerald-green secondary accent (#10B981) for positive results — is expressed throughout both the application and this report via JavaFX's “looked-up color” CSS variables, so the two themes share one set of component rules and differ only in their variable values.

6.2 Interaction Details

- **Live feedback** — the charset-detection chips, keyspace-size estimate, and strength meter all update on every keystroke via JavaFX property listeners, with no explicit “analyze” button required.
- **Background execution** — both the sequential and parallel passes run on a JavaFX Task on a background thread, with UI updates marshaled back to the JavaFX Application Thread via Platform.runLater(...), so the interface never freezes during a long search — a requirement the assignment explicitly calls out under application-quality grading criteria.
- **Input safety** — the Max Length spinner is restricted to digit input only via a JavaFX TextFormatter that silently rejects non-numeric keystrokes, with the committed value clamped to a safe range on focus loss.
- **Reveal/hide password** — implemented as a masked PasswordField and a plain TextField sharing one bound text property, with only one visible at a time; a small custom vector eye icon (an SVGPath with an overlaid diagonal Line) indicates the current state without any external icon-font dependency.
- **Motion** — a brief press animation on primary buttons, a fade-in on the results panel when a run completes, and a cross-fade when switching themes, implemented with JavaFX's javafx.animation API (ScaleTransition, FadeTransition).

7. Deployment

Deployment refers to the process of turning the project's Java source code into a form an ordinary Windows user can install and run without needing a Java Development Kit, a JavaFX SDK, or any awareness that the application is Java-based at all. This was accomplished in three conceptual stages: compiling the application into a runnable artifact, assembling a private, self-contained Java runtime around it, and wrapping that runtime in a native Windows installer.

7.1 From Source to Runnable Artifact

The project intentionally uses no external build tool (no Maven or Gradle); IntelliJ IDEA's own built-in build system compiles the source tree directly. The first deployment step packages the compiled classes and resources (FXML, CSS, images) into a single executable JAR — a zip-format archive with a manifest declaring the application's entry point (com.passgauge.Main) — produced via IntelliJ's Artifact configuration ("JAR from modules with dependencies"). This JAR is correct and runnable on any machine that already has a compatible JDK and the JavaFX SDK installed and configured — which is precisely the dependency the remaining steps exist to eliminate for an end user.

7.2 Building a Self-Contained Runtime Image

The jpackage tool, bundled with the JDK since Java 14, was used to remove that dependency entirely. Internally, jpackage invokes jlink — the Java Platform Module System's custom-runtime-image builder — to assemble a minimal, private Java Virtual Machine containing only the specific platform modules the application actually requires (java.base and the JavaFX modules javafx.controls and javafx.fxml), rather than bundling an entire general-purpose JRE. This step requires the JavaFX jmods distribution specifically (a modular packaging of JavaFX, distinct from the plain SDK jars used during development), supplied to jpackage via --module-path, because jlink operates on Java modules, not arbitrary classpath jars.

The result of this stage is an "app-image": a self-contained directory containing the application's own compiled code alongside a purpose-built, stripped-down JVM — large enough to run PassGauge, and nothing more. Anyone who later runs the installed application is, without realizing it, running this private runtime rather than any system-wide Java installation; this is what allows the finished installer to have zero external prerequisites.

7.3 Generating the Native Windows Installer

jpackage's --type exe target converts the app-image from the previous stage into an actual Windows installer executable, which internally requires the WiX Toolset (version 3) — jpackage shells out to WiX's own compiler and linker (candle.exe and light.exe) to author the Windows Installer (MSI-based) package definition and compile it into a distributable .exe setup wizard. This is why WiX is a deployment-time dependency on the build machine only — it plays no role at all on the end user's machine, which receives only the final compiled installer.

The command used to produce the final artifact was:

```
jpackage --type exe --name PassGauge --app-version 1.0 --vendor "..." \
  --input C:\PassGauge-dist --main-jar PassGauge.jar --main-class com.passgauge.Main \
  --module-path "C:\javafx-jmods-21" --add-modules javafx.controls,javafx.fxml \
  --icon packaging\PassGauge-icon.ico \
  --win-dir-chooser --win-shortcut --win-menu \
  --dest C:\PassGauge-release
```

The jpackage invocation that produced PassGauge-1.0.exe.

The resulting installer presents a standard Windows setup wizard: it lets the user choose an install directory (--win-dir-chooser), creates a Start Menu entry (--win-menu) and desktop shortcut (--win-shortcut), and embeds the application's icon (converted to the multi-resolution .ico format Windows installers require) at every one of those integration points. The installer was not digitally code-signed — a deliberate scope decision for a coursework deliverable, since code-signing requires a paid certificate from a recognized certificate authority; an unsigned installer will trigger a Windows SmartScreen warning on first run, which a user must explicitly dismiss ("More info → Run anyway") to proceed, but does not otherwise affect functionality.

7.4 Distribution

The finished installer is distributed via a GitHub Release attached to the project's public repository, rather than being committed into the repository's git history directly — binary installer files are large and do not diff meaningfully, so git history is reserved for source code, and compiled artifacts are instead tracked as versioned Release assets. The repository itself follows standard open-source conventions: a concise landing-page README summarizing the project with build-status badges, a separate SETUP.md containing the full build-from-source and packaging instructions, an MIT LICENSE file, and a .gitignore excluding IDE metadata and local build output so only source and documentation are tracked.

8. Testing and Verification

The screenshot below shows a representative benchmark run using the password *"zebra"* — a 5-character, all-lowercase word chosen specifically because it sits near the end of the alphabetical search order, forcing the sequential pass to traverse nearly the entire keyspace before finding it and giving a realistic, non-trivial timing comparison.

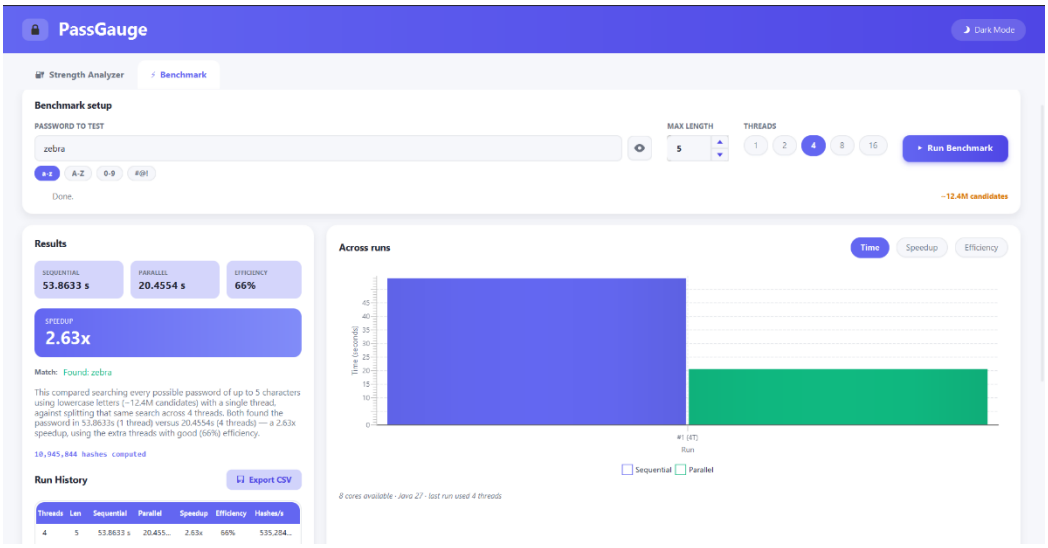


Figure 2 — Benchmark comparison for the password "zebra" (5 characters, lowercase, ~12.4M candidates): sequential vs. 4-thread parallel search, achieving a 2.63x speedup at 66% parallel efficiency.

With Max Length set to 5 and only lowercase letters detected, the search space totals approximately 12.4 million candidate passwords. Running single-threaded, the application took 53.8633 seconds to exhaustively search for and locate "zebra"; splitting that identical search across 4 threads reduced this to 20.4554 seconds — a 2.63x speedup.

Note that this falls short of the theoretical maximum of 4x (perfect linear scaling), which is reflected in the 66% parallel efficiency figure (2.63 ÷ 4 × 100). This gap is expected and consistent with Amdahl's Law: a portion of the work — managing the thread pool, and every worker checking the shared "found" flag between iterations — is inherently sequential overhead that cannot be parallelized away, so doubling or quadrupling the thread count never produces a proportional reduction in time. The system information line confirms the host machine has 8 cores available, meaning this run used only half the available parallelism (4 of 8 threads) — a comparison at 8 threads would be expected to show further speedup, though likely at a somewhat lower efficiency percentage still, per the same diminishing-returns pattern. In total, the parallel run performed 10,945,844 hash computations to locate the match, at a rate of roughly 535,284 hashes per second.

9. Conclusion

PassGauge satisfies every phase of the assignment brief: thread management is implemented with lock-free atomic coordination rather than blocking locks, eliminating both the specific race condition and the possibility of deadlock that the brief asks to be addressed explicitly; the chosen algorithm — SHA-256 keyspace brute force — exists in matched single-threaded and multi-threaded implementations differing only in thread count, so that the measured speedup reflects parallelism alone; and the algorithm is embedded in a complete, responsive desktop application that reports execution time for both versions, computes speedup and efficiency, and visualizes the result across repeated runs for direct inclusion in experimental analysis.

Beyond the assignment's core scope, the Strength Analyzer module demonstrates that the same thinking applied to how passwords are attacked can also inform how they are chosen in the first place — and does so under a strict, explicitly verified privacy constraint: nothing entered into that module is logged, stored, or transmitted. Taken together, the two modules make the case that fast brute force and informed password choice are two sides of the same underlying problem, examined through one application built around a single coherent theme.

References

- Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. AFIPS Spring Joint Computer Conference.
- Wheeler, D. L. (2016). zxcvbn: Low-Budget Password Strength Estimation. USENIX Security Symposium.
- Oracle. Java Platform, Standard Edition: java.util.concurrent and java.util.concurrent.atomic package documentation.
- Oracle. jlink — The Java Linker, and jpackage — Java Packaging Tool, JDK Tool Specifications.
- Gluon. JavaFX SDK and jmods distributions, gluonhq.com/products/javafx.
- WiX Toolset v3 documentation, wixtoolset.org.